Lab2：BombLab 实验报告

姓名：何思源　学号：24307090024

1. 各关实现思路

第一关

这一关主要看函数调用，`p1_compute` 连续调用三个小函数，最后把第三步的结果与第一步的结果异或。相比后面的题，这一关思路比较简单，把每次传入的参数和返回值弄清楚，就能把计算过程接起来。下面几条指令分别对应三个函数里的计算，其中 `lea` 可以理解成这里的加法和乘法。

```
p1_step0:
  lea 0x5926(%rsi),%eax
  xor %edi,%eax
p1_step1:
  sub %esi,%edi
  lea 0xbc40(%rdi,%rdi,2),%eax
p1_step2:
  lea (%rdi,%rsi,4),%eax
  xor $0xada4,%eax
```

根据调用前给寄存器赋的值，先分别取出 challenge 的低 32 位、高 32 位和右移 16 位后的低 32 位，再按三个函数的顺序计算。第一步的结果保存在 `%ecx` 中，在最后一条 `xor %ecx,%eax` 中用到，对应的 C 代码如下。

```
uint32_t low = (uint32_t)c;
uint32_t high = (uint32_t)(c >> 32);
uint32_t middle = (uint32_t)(c >> 16);
uint32_t t0 = (high + 0x5926u) ^ low;
uint32_t t1 = 3u * (high - middle) + 0xbc40u;
uint32_t t2 = (t1 + 4u * low) ^ 0xada4u;
uint32_t answer = t2 ^ t0;
```

第二关

这一关需要六个整数，先根据 challenge 算出第一个，后面的每个数都由前一个数递推得到。循环里的关键部分如下，`lea (%rsi,%rsi,2),%ecx` 就是将前一个数乘以 3。

```
lea 0xf(%rdx),%ecx
mov %r13,%rax
shr %cl,%rax
lea (%rsi,%rsi,2),%ecx
and $0xf,%eax
xor %edi,%eax
add %ecx,%eax
```

首项通过右移 22 位、取低五位并与 23 异或得到，循环中的 %rdx 每次加 4，所以后面五次右移量是 15、19、23、27、31。这里感觉比较容易看错，数组下标每次增加 1，但汇编中的字节偏移增加的是 4，移位量也跟着变化。%edi 则从 11 开始每次加 1，将这个过程写成 C 循环如下。

```
uint32_t v[6];
v[0] = ((c >> 22) & 31) ^ 23;
for (int i = 1; i < 6; i++) {
    int shift = 15 + 4 * (i - 1);
    uint32_t digit = (c >> shift) & 15;
    v[i] = 3 * v[i - 1] + (digit ^ (10 + i));
}
```

第三关

这一关和 switch 分支有关，需要输出分支编号和该分支算出的值。汇编里通过下面的间接跳转选择分支，每个表项占 8 字节。

```
cmp $0x7,%eax
notrack jmp *0x405820(,%rax,8)
```

前面的 and $0x7,%edi 得到编号 k=c&7，传给分支函数的参数是 x=u32(c>>3)^u32(c>>32)。这一关挺有意思，C 里的 switch 在汇编里变成了按地址跳转，而且表里的顺序和函数名字中的编号还不同，直接按名字理解就容易弄混。按实际跳转目标整理后得到下表，最后输出 k 和第二列的结果。

k	计算结果
0	5*x + 0x148a
1	3*x + 0x574d
2	5*x + 0xbce3
3	(4*x) ^ 0xf7a2
4	(x ^ 0x3775) + (x>>2)
5	(4*x) ^ 0x30f5
6	5*x + 0x12f70
7	(x ^ 0x4ea) + (x>>2)

第四关

这一关用到了递归，p4_walk 每次取区间中点，再根据目标值选择左边或右边，过程和二分查找比较像。返回时还要继续计算，下面两条指令表示把递归结果乘以 5，再和当前层保存的值异或。

```
call p4_walk
lea (%rax,%rax,4),%eax
xor %ebx,%eax
```

初始区间为 [0,127]，目标是 target=c&127，另一个固定参数是 key=u32(c>>7)^u32(c>>32)。每次先计算中点 mid=low+((high-low)>>1)，目标等于中点时，返回 (mid+0x93)^key，这一层就结束了。

目标小于中点时，继续在 [low,mid-1] 内递归，拿到返回值后乘以 5，再与 mid+key 和 0x0c 异或。目标大于中点时，改为在 [mid+1,high] 内递归，最后的常量也改成 0x08。这一关对我来说比较难，递归往下找的过程还能理解，返回时每层还要继续算就比较绕，这部分借助了 AI 解释。按照这个过程写递归函数，从初始区间调用得到的返回值就是本关口令。

第五关

这一关需要沿链表走，输出累计结果和最后一个节点的编号。节点里的三个字段分别在偏移 0、4、8 的位置，从下面的访问可以看出，前两个字段是 32 位整数，最后一个是指向下一节点的指针。

```
mov 0x4(%rdx),%esi
xor (%rdx),%eax
add %esi,%eax
mov 0x8(%rdx),%rdx
```

因此每个节点可表示成 value、id、next，占 16 字节。以地址 0x405ba0 的节点为 0，按地址递增编号，读取头指针表得到起点顺序 [6,2,1,10,11,4,3,0]。程序使用 c&7 选择起点，先处理这个节点，再沿 next 移动 (c>>3)&3 次。

累计值从 u32(c>>6)^u32(c>>32)^0x6da3 开始，每访问一个节点就计算 acc=(acc^value)+id，同时保存这个节点的 id。这一关的难点在于先看懂节点里的数据，借助 AI 把字段和指针关系整理出来后，后面的遍历和普通链表题就比较接近了。实现中把读出的固定节点数据和连接关系放入数组，用下标模拟指针，最终输出 acc 和最后保存的 id。

第六关

这一关也是查表，但表里存的是函数地址，前一次算出的值还会传给下一次调用。关键指令如下，中间的异或和最后的加法也属于口令计算。

```
call *0x405e00(,%rax,8)
xor $0xed66,%eax
mov %eax,%edi
call *0x405e00(%rbx)
add $0x8b77,%eax
```

两个索引分别是 i=c&7 和 j=(c>>3)&7，初始参数为 x=u32(c>>6)^u32(c>>32)。读取函数指针表并查看各个函数后，可以按表中的顺序写出操作 G。

索引	G 的计算结果
0	`(x ^ 0xc4fd) + (x<<7)`
1	`((x<<5) + 0xd14c) ^ (5*x)`
2、4	`(x + 0xac92) ^ (x>>3)`
3、7	`((x>>2) ^ x) + 0x151c`
5	`(x>>2) | (x<<30)`
6	`((x<<1) | (x>>31)) ^ 0xd2c2`

其中两组索引指向相同函数，索引 5 和 6 还涉及循环移位，按无符号 32 位整数实现即可。这题和第三关有相似之处，都是先查表确定接下来执行什么，只是这里查到的是函数地址，而且要连续调用两次。先算 `t=G(i,x)^0xed66`，再输出 `G(j,t)+0x8b77`，就对应了两次调用的顺序。

Honor

Honor 除了传入 challenge，还会收到前六关答案生成的 `context`，这关需要先处理输入的布局。`secret_gate` 把整行复制到 `rbp-0xd0`，随后检查 `rbp-0xb0` 的值，并从 `rbp-0xac` 开始读取三个整数。

```
call secret_copy
cmpl $0x4e45504f,-0xb0(%rbp)
...
lea -0xac(%rbp),%rdi
call secret_phase
```

两个位置与复制起点分别相差 32 和 36 字节，`0x4e45504f` 按小端顺序对应 `OPEN`。这部分比前面的整数计算更难理解，我借助 AI 分析了这几个地址的关系，才弄清楚前面还需要填充字符。先写 32 个 `A`，接着写 `OPEN`，最后接上三个用空格分开的整数，整数部分正好从偏移 36 开始。

再看三个整数的计算，`secret_mix` 中的 `ror $0x7,%eax` 表示循环右移 7 位，接着与 `0x9a94` 异或。先将 challenge 和 context 各自的高低 32 位异或，再结合后面的比较指令，可以写出下面这段计算代码。

```
uint32_t a = (uint32_t)c ^ (uint32_t)(c >> 32);
uint32_t b = (uint32_t)context ^ (uint32_t)(context >> 32);
uint32_t x = a ^ b;
uint32_t rotated = (x >> 7) | (x << 25);
uint32_t v0 = (uint32_t)context & 3u;
uint32_t v1 = rotated ^ 0x9a94u ^ (v0 * 0xe940u)
              ^ (uint32_t)(context >> 16);
uint32_t v2 = v1 ^ (b + v0 + 0x73d4u + (a >> 7));
```

代码中的 `rotated` 对应循环右移，把算出的 `v0`、`v1` 和 `v2` 用空格分开放在填充和标记之后，就得到完整第七行。入口允许的长度为 36 到 163 个字符，这个构造最长为 68 个字符，符合长度要求。

2. 测试

在 ~/ICS/BombLab 中执行 make 编译，通过后运行公开测试，六个正式关卡通过 60/60，Honor 通过 10/10。截图时用 grep 保留汇总，方便查看结果。

```
python3 public_test.py | grep -E '^(Public tests:|Secret public tests:)'
```

另外选取公开列表之外的 seed 24307090024，将六关口令和 Honor 的第七行一起送给炸弹，七关都显示通关。下面的命令保留了每关的成功提示，同一张截图也包含前面的公开测试结果。

```
{ ./defuser --seed 24307090024 && ./defuser --secret --seed 24307090024; } \
  | ./bomb --seed 24307090024 \
  | grep -E '^Phase [1-6] defused\.|^Secret phase defused\.'
```



3. 资料与工具使用

实验参考了课程说明和仓库 README，用来确认领取方法、函数接口和测试要求。计算关系来自个人炸弹的 objdump 输出，驱动中的 challenge.c 和 defuser_main.c 用于核对参数与 context 的来源。同时，这次Lab使用了 Codex 辅助分析汇编，提供思路。